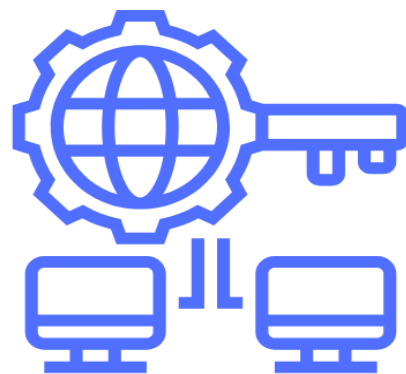
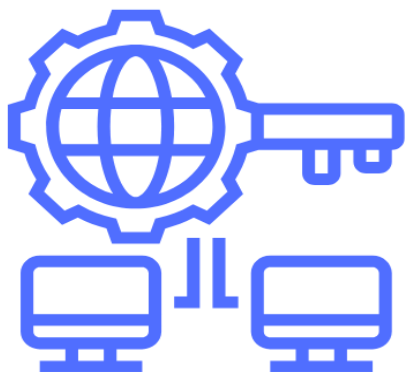




ESTUDI COMPLEXITAT

Pol Sànchez i Arnau Briñas

2025- 2026



ÍNDEX

1. ArbolBST — Arbre Binari de Cerca.....	3
2. Distancia de Levenshtein.....	3
3. FiltroPorPalabrasClave.analizar().....	4
4. FiltroPorRemitenteSospechoso.analizar().....	5
5. AnalizadorCorreo, cost global per correu.....	5
6. escanear_carpeta(), escaneig recursiu.....	6

1. ArbolBST — Arbre Binari de Cerca

Per gestionar les paraules sospitoses, hem implementat un arbre binari de cerca (BST) que actua com a diccionari intern del sistema. Cada node de l'arbre representa una paraula del diccionari PALABRAS_SPAM i emmagatzema, a més, el pes de risc associat i el nombre de vegades que ha aparegut al correu analitzat.

Operació	Cas millor	Cas mitjà	Cas pitjor
insertar()	$O(\log n)$	$O(\log n)$	$O(n)$
buscar()	$O(1)$	$O(\log n)$	$O(n)$
calcular_riesgo_total()	$O(n)$	$O(n)$	$O(n)$
palabras_encontradas()	$O(n)$	$O(n)$	$O(n)$

Cal tenir en compte que, en el cas pitjor, l'arbre pot degenerar en una llista enllaçada si les paraules s'insereixen en ordre alfabètic estricte, cosa que elevaria el cost de les operacions a $O(n)$. Tot i això, com que el diccionari és fix i té només 47 paraules, aquest escenari no té cap impacte rellevant en el rendiment real del sistema.

Pel que fa a la complexitat espacial, és $O(n)$ on n és el nombre de paraules úniques inserides. Cada node ocupa un espai constant, ja que només emmagatzema la paraula, el seu pes de risc, la freqüència d'aparició i dos punters als fills.

2. Distancia de Levenshtein

La funció `distancia_levenshtein()` s'encarrega de detectar el que s'anomena typosquatting, és a dir, dominis falsos que imiten als legítims canviant algun caràcter per exemple, santanderr.com en lloc de santander.com. Per fer-ho, implementa aquest algoritme que calcula quantes operacions, ja sigui insercions, esborrats o substitucions calen per transformar una cadena en l'altra.

Paràmetre	Valor
Complexitat temporal	$O(m \times n)$
Complexitat espacial	$O(m \times n)$
m	Longitud del domini analitzat
n	Longitud del domini legítim
Crides per correu	$O(d \times L)$ on d dominis, L dominis legítics

Amb els valors reals del projecte, tenim 22 dominis legítics i una longitud mitjana de domini d'uns 12 caràcters, de manera que cada crida a la funció té un cost de $O(12 \times 12) = O(144)$. Si un correu conté d dominis extrets, el cost total serà $O(d \times 22 \times 144)$, que per a correus normals és assumible.

3. FiltroPorPalabrasClave.analizar()

Aquest filtre recorre totes les 47 entrades del diccionari PALABRAS_SPAM i per cadascuna comprova si apareix al text del correu utilitzant l'operador in de Python, que funciona com una cerca de subcadena.

Pel que fa al cost, si anomenem T la longitud del text del correu i P la longitud de la paraula que estem buscant, cada comprovació individual té un cost de $O(T + P)$. Com que ho fem per les 47 paraules, el cost total del filtre queda $O(k \times (T + P_{\text{max}}))$, on $k = 47$ i P_{max} és la longitud de la paraula més llarga del diccionari.

Quant a l'espai, només cal guardar les paraules que s'han trobat realment al text, de manera que en el pitjor cas serà $O(k)$, amb $k \leq 47$.

4. FiltroPorRemitenteSospechoso.analizar()

Aquest filtre comença extraient tots els dominis que apareixen al text mitjançant expressions regulars (regex), i després aplica tres comprovacions per a cadascun.

Comprovació	Cost per domini	Cost total
Llista negra (in)	$O(B)$	$O(d \times B)$
Typosquatting (Levenshtein)	$O(L \times m \times n)$	$O(d \times L \times m \times n)$
URL amb @ (regex)	$O(T)$	$O(T)$

$B = 8$ dominis a la llista negra, $L = 22$ legítims, $d =$ dominis extrets del correu, $m, n =$ longituds de dominis.

5. AnalizadorCorreo, cost global per correu

L'analitzador simplement aplica els dos filtres un darrere l'altre, de manera que el cost total és la suma dels dos. D'una banda, el filtre de paraules clau té un cost de $O(k \times T)$, és a dir, busca k paraules sospitoses dins d'un text de longitud T . De l'altra, el filtre de dominis té un cost de $O(d \times L \times m \times n)$, on per a cada un dels d dominis trobats al correu s'aplica Levenshtein contra els L dominis legítims, amb un cost de $m \times n$ per cada comparació. Sumant-ho tot queda $O(k \times T + d \times L \times m \times n)$.

6. escanear_carpeta(), escaneig recursiu

La funció recorre recursivament tota l'estructura de directoris, és a dir, entra a cada carpeta i, si troba subcarpetes, també hi entra, i així fins arribar al fons. Si anomenem C el nombre total de correus .eml que hi ha en total i D la profunditat màxima de carpetes anidades, el cost queda de la manera següent:

Operació	Complexitat
Recorregut de fitxers i carpetes	$O(\text{fitxers} + \text{carpetes})$
Anàlisi de tots els correus	$O(C \times (k \times T + d \times L \times m \times n))$
Complexitat total	$O(C \times (k \times T + d \times L \times m \times n))$